

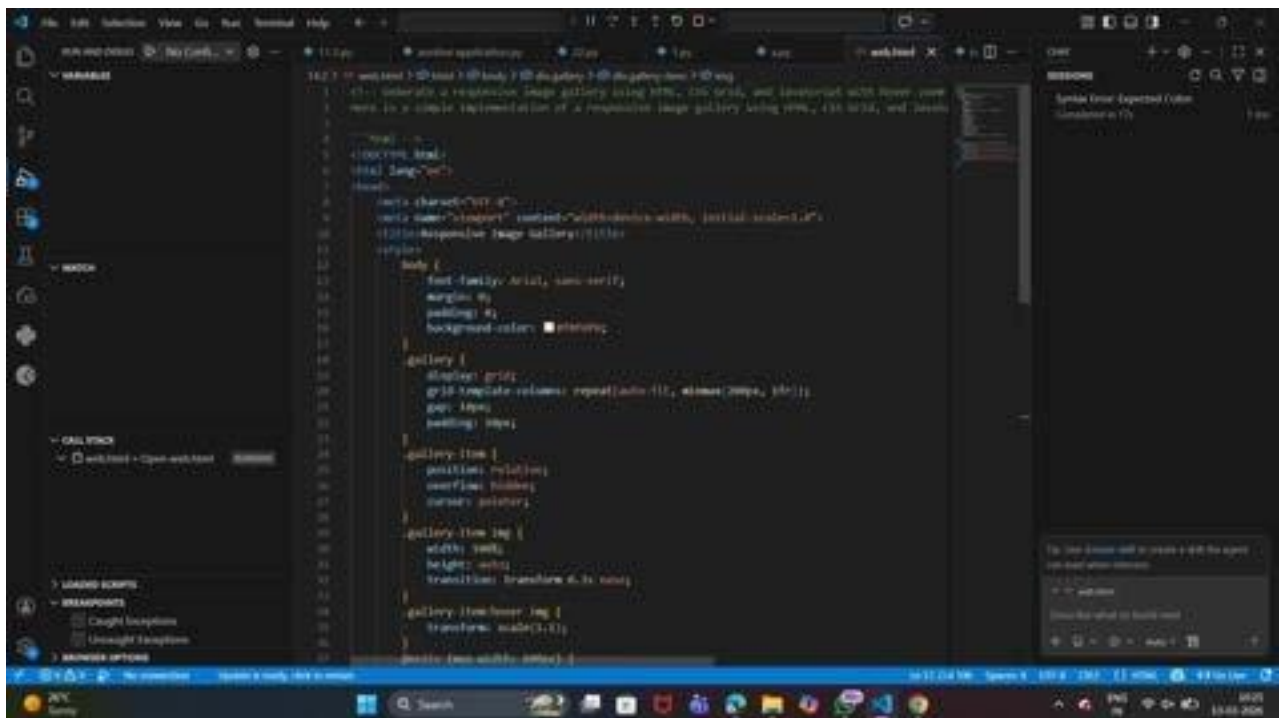
Batch-51

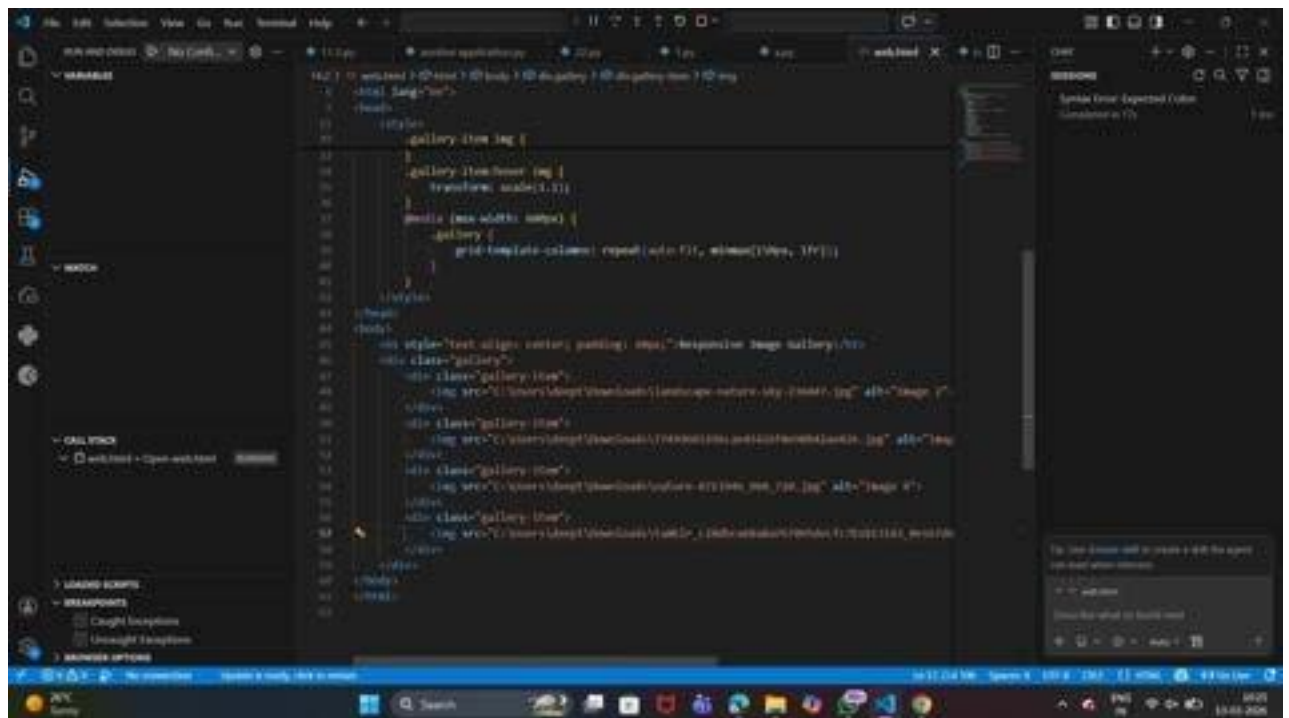
Web Design Application – AI-Assisted HTML/CSS/JS Generation

Task Description #1 (Image Gallery Website):

PROMPT: Generate a responsive image gallery using HTML, CSS Grid, and JavaScript with hover zoom effects and mobile-friendly layout.

CODE:





OUTPUT:



Explanation

- CSS Grid arranges images in a structured grid.
- Hover zoom effect improves visual interaction.
- Responsive design adjusts layout for different devices.

Task Description #2 (Profile Card Generator):

PROMPT: Create a modern profile card using HTML, CSS Flexbox, and JavaScript showing user image, name, role, and social media links with hover animation.

CODE:

```

1 // Create a modern profile card using HTML, CSS Flexbox, and JavaScript showing user image,
2 // name, role, and social media links.
3 // Add hover zoom effect and responsive design.
4 // Here is a simple implementation of a modern profile card using HTML, CSS Flexbox, and JavaScript.
5
6 // HTML
7
8 // CSS
9
10 // JavaScript
11
12 // HTML
13
14 // CSS
15
16 // JavaScript
17
18 // HTML
19
20 // CSS
21
22 // JavaScript
23
24 // HTML
25
26 // CSS
27
28 // JavaScript
29
30 // HTML
31
32 // CSS
33
34 // JavaScript
35
36 // HTML
37
38 // CSS
39
40 // JavaScript
41
42 // HTML
43
44 // CSS
45
46 // JavaScript
47
48 // HTML
49
50 // CSS
51
52 // JavaScript
53
54 // HTML
55
56 // CSS
57
58 // JavaScript
59
60 // HTML
61
62 // CSS
63
64 // JavaScript
65
66 // HTML
67
68 // CSS
69
70 // JavaScript
71
72 // HTML
73
74 // CSS
75
76 // JavaScript
77
78 // HTML
79
80 // CSS
81
82 // JavaScript
83
84 // HTML
85
86 // CSS
87
88 // JavaScript
89
90 // HTML
91
92 // CSS
93
94 // JavaScript
95
96 // HTML
97
98 // CSS
99
100 // JavaScript
101
102 // HTML
103
104 // CSS
105
106 // JavaScript
107
108 // HTML
109
110 // CSS
111
112 // JavaScript
113
114 // HTML
115
116 // CSS
117
118 // JavaScript
119
120 // HTML
121
122 // CSS
123
124 // JavaScript
125
126 // HTML
127
128 // CSS
129
130 // JavaScript
131
132 // HTML
133
134 // CSS
135
136 // JavaScript
137
138 // HTML
139
140 // CSS
141
142 // JavaScript
143
144 // HTML
145
146 // CSS
147
148 // JavaScript
149
150 // HTML
151
152 // CSS
153
154 // JavaScript
155
156 // HTML
157
158 // CSS
159
160 // JavaScript
161
162 // HTML
163
164 // CSS
165
166 // JavaScript
167
168 // HTML
169
170 // CSS
171
172 // JavaScript
173
174 // HTML
175
176 // CSS
177
178 // JavaScript
179
180 // HTML
181
182 // CSS
183
184 // JavaScript
185
186 // HTML
187
188 // CSS
189
190 // JavaScript
191
192 // HTML
193
194 // CSS
195
196 // JavaScript
197
198 // HTML
199
200 // CSS
201
202 // JavaScript
203
204 // HTML
205
206 // CSS
207
208 // JavaScript
209
210 // HTML
211
212 // CSS
213
214 // JavaScript
215
216 // HTML
217
218 // CSS
219
220 // JavaScript
221
222 // HTML
223
224 // CSS
225
226 // JavaScript
227
228 // HTML
229
230 // CSS
231
232 // JavaScript
233
234 // HTML
235
236 // CSS
237
238 // JavaScript
239
240 // HTML
241
242 // CSS
243
244 // JavaScript
245
246 // HTML
247
248 // CSS
249
250 // JavaScript
251
252 // HTML
253
254 // CSS
255
256 // JavaScript
257
258 // HTML
259
260 // CSS
261
262 // JavaScript
263
264 // HTML
265
266 // CSS
267
268 // JavaScript
269
270 // HTML
271
272 // CSS
273
274 // JavaScript
275
276 // HTML
277
278 // CSS
279
280 // JavaScript
281
282 // HTML
283
284 // CSS
285
286 // JavaScript
287
288 // HTML
289
290 // CSS
291
292 // JavaScript
293
294 // HTML
295
296 // CSS
297
298 // JavaScript
299
300 // HTML
301
302 // CSS
303
304 // JavaScript
305
306 // HTML
307
308 // CSS
309
310 // JavaScript
311
312 // HTML
313
314 // CSS
315
316 // JavaScript
317
318 // HTML
319
320 // CSS
321
322 // JavaScript
323
324 // HTML
325
326 // CSS
327
328 // JavaScript
329
330 // HTML
331
332 // CSS
333
334 // JavaScript
335
336 // HTML
337
338 // CSS
339
340 // JavaScript
341
342 // HTML
343
344 // CSS
345
346 // JavaScript
347
348 // HTML
349
350 // CSS
351
352 // JavaScript
353
354 // HTML
355
356 // CSS
357
358 // JavaScript
359
360 // HTML
361
362 // CSS
363
364 // JavaScript
365
366 // HTML
367
368 // CSS
369
370 // JavaScript
371
372 // HTML
373
374 // CSS
375
376 // JavaScript
377
378 // HTML
379
380 // CSS
381
382 // JavaScript
383
384 // HTML
385
386 // CSS
387
388 // JavaScript
389
390 // HTML
391
392 // CSS
393
394 // JavaScript
395
396 // HTML
397
398 // CSS
399
400 // JavaScript
401
402 // HTML
403
404 // CSS
405
406 // JavaScript
407
408 // HTML
409
410 // CSS
411
412 // JavaScript
413
414 // HTML
415
416 // CSS
417
418 // JavaScript
419
420 // HTML
421
422 // CSS
423
424 // JavaScript
425
426 // HTML
427
428 // CSS
429
430 // JavaScript
431
432 // HTML
433
434 // CSS
435
436 // JavaScript
437
438 // HTML
439
440 // CSS
441
442 // JavaScript
443
444 // HTML
445
446 // CSS
447
448 // JavaScript
449
450 // HTML
451
452 // CSS
453
454 // JavaScript
455
456 // HTML
457
458 // CSS
459
460 // JavaScript
461
462 // HTML
463
464 // CSS
465
466 // JavaScript
467
468 // HTML
469
470 // CSS
471
472 // JavaScript
473
474 // HTML
475
476 // CSS
477
478 // JavaScript
479
480 // HTML
481
482 // CSS
483
484 // JavaScript
485
486 // HTML
487
488 // CSS
489
490 // JavaScript
491
492 // HTML
493
494 // CSS
495
496 // JavaScript
497
498 // HTML
499
500 // CSS
501
502 // JavaScript
503
504 // HTML
505
506 // CSS
507
508 // JavaScript
509
510 // HTML
511
512 // CSS
513
514 // JavaScript
515
516 // HTML
517
518 // CSS
519
520 // JavaScript
521
522 // HTML
523
524 // CSS
525
526 // JavaScript
527
528 // HTML
529
530 // CSS
531
532 // JavaScript
533
534 // HTML
535
536 // CSS
537
538 // JavaScript
539
540 // HTML
541
542 // CSS
543
544 // JavaScript
545
546 // HTML
547
548 // CSS
549
550 // JavaScript
551
552 // HTML
553
554 // CSS
555
556 // JavaScript
557
558 // HTML
559
560 // CSS
561
562 // JavaScript
563
564 // HTML
565
566 // CSS
567
568 // JavaScript
569
570 // HTML
571
572 // CSS
573
574 // JavaScript
575
576 // HTML
577
578 // CSS
579
580 // JavaScript
581
582 // HTML
583
584 // CSS
585
586 // JavaScript
587
588 // HTML
589
590 // CSS
591
592 // JavaScript
593
594 // HTML
595
596 // CSS
597
598 // JavaScript
599
600 // HTML
601
602 // CSS
603
604 // JavaScript
605
606 // HTML
607
608 // CSS
609
610 // JavaScript
611
612 // HTML
613
614 // CSS
615
616 // JavaScript
617
618 // HTML
619
620 // CSS
621
622 // JavaScript
623
624 // HTML
625
626 // CSS
627
628 // JavaScript
629
630 // HTML
631
632 // CSS
633
634 // JavaScript
635
636 // HTML
637
638 // CSS
639
640 // JavaScript
641
642 // HTML
643
644 // CSS
645
646 // JavaScript
647
648 // HTML
649
650 // CSS
651
652 // JavaScript
653
654 // HTML
655
656 // CSS
657
658 // JavaScript
659
660 // HTML
661
662 // CSS
663
664 // JavaScript
665
666 // HTML
667
668 // CSS
669
670 // JavaScript
671
672 // HTML
673
674 // CSS
675
676 // JavaScript
677
678 // HTML
679
680 // CSS
681
682 // JavaScript
683
684 // HTML
685
686 // CSS
687
688 // JavaScript
689
690 // HTML
691
692 // CSS
693
694 // JavaScript
695
696 // HTML
697
698 // CSS
699
700 // JavaScript
701
702 // HTML
703
704 // CSS
705
706 // JavaScript
707
708 // HTML
709
710 // CSS
711
712 // JavaScript
713
714 // HTML
715
716 // CSS
717
718 // JavaScript
719
720 // HTML
721
722 // CSS
723
724 // JavaScript
725
726 // HTML
727
728 // CSS
729
730 // JavaScript
731
732 // HTML
733
734 // CSS
735
736 // JavaScript
737
738 // HTML
739
740 // CSS
741
742 // JavaScript
743
744 // HTML
745
746 // CSS
747
748 // JavaScript
749
750 // HTML
751
752 // CSS
753
754 // JavaScript
755
756 // HTML
757
758 // CSS
759
760 // JavaScript
761
762 // HTML
763
764 // CSS
765
766 // JavaScript
767
768 // HTML
769
770 // CSS
771
772 // JavaScript
773
774 // HTML
775
776 // CSS
777
778 // JavaScript
779
780 // HTML
781
782 // CSS
783
784 // JavaScript
785
786 // HTML
787
788 // CSS
789
790 // JavaScript
791
792 // HTML
793
794 // CSS
795
796 // JavaScript
797
798 // HTML
799
800 // CSS
801
802 // JavaScript
803
804 // HTML
805
806 // CSS
807
808 // JavaScript
809
810 // HTML
811
812 // CSS
813
814 // JavaScript
815
816 // HTML
817
818 // CSS
819
820 // JavaScript
821
822 // HTML
823
824 // CSS
825
826 // JavaScript
827
828 // HTML
829
830 // CSS
831
832 // JavaScript
833
834 // HTML
835
836 // CSS
837
838 // JavaScript
839
840 // HTML
841
842 // CSS
843
844 // JavaScript
845
846 // HTML
847
848 // CSS
849
850 // JavaScript
851
852 // HTML
853
854 // CSS
855
856 // JavaScript
857
858 // HTML
859
860 // CSS
861
862 // JavaScript
863
864 // HTML
865
866 // CSS
867
868 // JavaScript
869
870 // HTML
871
872 // CSS
873
874 // JavaScript
875
876 // HTML
877
878 // CSS
879
880 // JavaScript
881
882 // HTML
883
884 // CSS
885
886 // JavaScript
887
888 // HTML
889
890 // CSS
891
892 // JavaScript
893
894 // HTML
895
896 // CSS
897
898 // JavaScript
899
900 // HTML
901
902 // CSS
903
904 // JavaScript
905
906 // HTML
907
908 // CSS
909
910 // JavaScript
911
912 // HTML
913
914 // CSS
915
916 // JavaScript
917
918 // HTML
919
920 // CSS
921
922 // JavaScript
923
924 // HTML
925
926 // CSS
927
928 // JavaScript
929
930 // HTML
931
932 // CSS
933
934 // JavaScript
935
936 // HTML
937
938 // CSS
939
940 // JavaScript
941
942 // HTML
943
944 // CSS
945
946 // JavaScript
947
948 // HTML
949
950 // CSS
951
952 // JavaScript
953
954 // HTML
955
956 // CSS
957
958 // JavaScript
959
960 // HTML
961
962 // CSS
963
964 // JavaScript
965
966 // HTML
967
968 // CSS
969
970 // JavaScript
971
972 // HTML
973
974 // CSS
975
976 // JavaScript
977
978 // HTML
979
980 // CSS
981
982 // JavaScript
983
984 // HTML
985
986 // CSS
987
988 // JavaScript
989
990 // HTML
991
992 // CSS
993
994 // JavaScript
995
996 // HTML
997
998 // CSS
999
1000 // JavaScript

```

OUTPUT:



Explanation

- Flexbox aligns card elements properly.
- Modern colors and fonts improve UI design.
- Hover animation adds interactive effects.

Task Description #3 (Develop a dynamic shopping cart system):

Prompt : Develop a dynamic shopping cart using HTML, CSS, and JavaScript that allows adding, removing, and updating items with real-time total price and localStorage support.

Code :

Task description #4 (Role-Based Access Control Application):

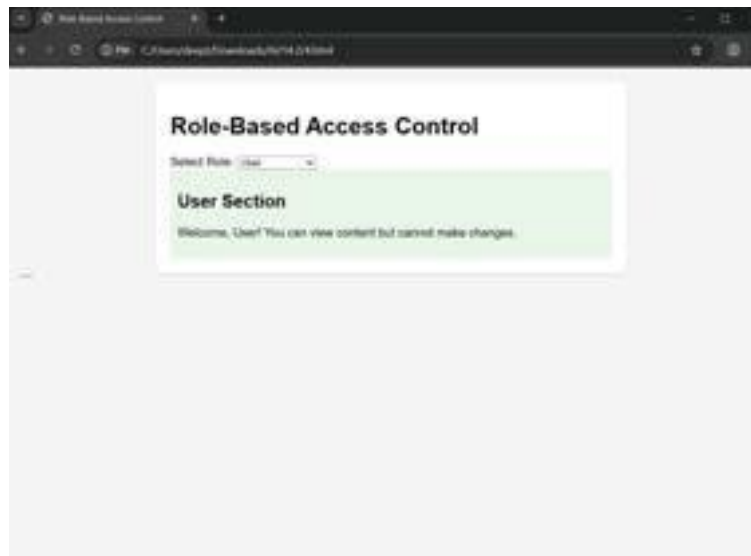
Prompt : Create a role-based web application using HTML, CSS, and JavaScript with Admin, Editor, and User roles and conditional UI rendering and Design access control logic, Maintain secure state transitions, Create scalable UI components.

Code :

```
142 > 0.4.html >
1 1<!-- Create a role-based access control (RBAC) web app using HTML, CSS, and JavaScript.
2 Roles: Admin, Editor, User.
3 Show different UI sections based on role using conditional rendering.
4 Here is a simple implementation of a Role-based access control (RBAC) web app using HTML, CSS,
5 and JavaScript.
6 <!DOCTYPE html>
7 <html lang="en">
8 <head>
9   <meta charset="UTF-8">
10  <meta name="viewport" content="width=device-width, initial-scale=1.0">
11  <title>Role-Based Access Control</title>
12  <style>
13    body {
14      font-family: Arial, sans-serif;
15      background-color: #f0f0f0;
16      margin: 0;
17      padding: 20px;
18    }
19    .container {
20      max-width: 600px;
21      margin: auto;
22      background-color: #fff;
23      padding: 20px;
24      border-radius: 5px;
25      box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
26    }
27    .section {
28      display: none;
29    }
30    .admin-section {
31      background-color: #e0f2f1;
32      padding: 10px;
33      margin-bottom: 20px;
34    }
35    .editor-section {
36      background-color: #e0f2f1;
37      padding: 10px;
38    }
39  </style>
40</head>
```

```
142 > 0.4.html >
41  <body>
42    <div class="container">
43      <select id="role" onchange="renderSections()">
44        <option value="">Select Role</option>
45        <option value="admin">Admin</option>
46        <option value="editor">Editor</option>
47        <option value="user">User</option>
48      </select>
49
50      <div class="section admin-section" id="adminSection">
51        <h3>Admin Section</h3>
52        <p>Welcome, Admin! You have full access to the system.</p>
53      </div>
54
55      <div class="section editor-section" id="editorSection">
56        <h3>Editor Section</h3>
57        <p>Welcome, Editor! You can edit content but have limited access to settings.</p>
58      </div>
59
60      <div class="section user-section" id="userSection">
61        <h3>User Section</h3>
62        <p>Welcome, User! You can view content but cannot make changes.</p>
63      </div>
64    </div>
65
66    <script>
67      function renderSections() {
68        const role = document.getElementById('role').value;
69        document.getElementById('adminSection').style.display = role === 'admin' ? 'block' : 'none';
70        document.getElementById('editorSection').style.display = role === 'editor' ? 'block' : 'none';
71        document.getElementById('userSection').style.display = role === 'user' ? 'block' : 'none';
72      }
73    </script>
74  </body>
75</html>
```

Output :



Explanation

- Different roles have different permissions.
- UI changes based on user role.
- Improves security and access control.

Task Description #5 (Multi-Language Website-Design a multilingual website):

Prompt : Design a multilingual website using HTML, CSS, and JavaScript with dynamic language switching and language preference stored in localStorage and Structure translation files,Handle RTL/LTR layouts,Improve accessibility.

Code :


```

142 > @html > @html > @html > @style > % container
1  <!-- Develop a multilingual website using HTML, CSS, and JavaScript.
2  Allow dynamic language switching.
3  Store translations in JSON and save language preference in localStorage.
4  Here is a simple implementation of a multilingual website using HTML, CSS, and JavaScript that
5  ...
6  <doctype html>
7  <html lang="en">
8  <head>
9      <meta charset="UTF-8">
10     <meta name="viewport" content="width=device-width, initial-scale=1.0">
11     <title>Multilingual Website</title>
12     <style>
13         body {
14             font-family: Arial, sans-serif;
15             background-color: #f0f0f0;
16             margin: 0;
17             padding: 20px;
18         }
19         .container {
20             max-width: 600px;
21             margin: auto;
22             background-color: #fff;
23             padding: 20px;
24             border-radius: 4px;
25             box-shadow: 0 2px 4px #888;
26         }
27         .language-selector {
28             margin-bottom: 20px;
29         }
30     </style>
31 </head>
32 <body>
33     <div class="container">
34         <h1 id="title">Welcome to our website!</h1>
35         <p id="description">This is a simple multilingual website example.</p>
36         <div class="language-selector">
37             <button class="switch-language">Switch Language</button>

```

```

142 > @html > @html > @html > @html > @style > % container
7  <html lang="en">
12 <body>
13     <div class="container">
14     </div>
15
16     <script>
17         const translations = {
18             en: {
19                 title: "Welcome to our website!",
20                 description: "This is a simple multilingual website example."
21             },
22             es: {
23                 title: "¡Bienvenido a nuestro sitio web!",
24                 description: "Este es un ejemplo de sitio web multilingüe simple."
25             },
26             fr: {
27                 title: "Bienvenue sur notre site web!",
28                 description: "Ceci est un exemple de site web multilingue simple."
29             }
30         };
31
32         function switchLanguage() {
33             const language = document.getElementById("language").value;
34             document.getElementById("title").textContent = translations[language].title;
35             document.getElementById("description").textContent = translations[language].description;
36             localStorage.setItem("preferredLanguage", language);
37         }
38
39         // Load preferred language on page load
40         window.onload = function() {
41             const preferredLanguage = localStorage.getItem("preferredLanguage") || "en";
42             document.getElementById("language").value = preferredLanguage;
43             switchLanguage();
44         };
45     </script>
46 </body>
47 </html>

```

Output :



Explanation:

- JSON files store translation data.
- Users can switch languages dynamically.
- Language preference is saved for future visits.

Task Description #6 (Online Examination System-Create an online exam interface.):

Prompt : Create an online examination interface using HTML, CSS, and JavaScript with MCQ questions, countdown timer, and single submission system and Manage exam state,Improve exam UX,Ensure accessibility.

Code :

```

142 < @head> @head> @head> @style ? % container
143 < @html lang="en">
144 < @body>
145
146 < /style>
147 < /head>
148 < body>
149
150 < div class="container">
151 < @Online Examination />
152 < @div class="timer" id="timer">Time Remaining: 41:00 />
153 < @div class="question">
154 < @p1, What is the capital of France?>
155 < @div class="options">
156 < @div class="option">< input type="radio" name="q1" value="A"> A) Berlin />
157 < @div class="option">< input type="radio" name="q1" value="B"> B) Madrid />
158 < @div class="option">< input type="radio" name="q1" value="C"> C) Paris />
159 < @div class="option">< input type="radio" name="q1" value="D"> D) Rome />
160 < /div>
161 < /div>
162 < @div class="question">
163 < @p2, Which planet is known as the Red Planet?>
164 < @div class="options">
165 < @div class="option">< input type="radio" name="q2" value="A"> A) Earth />
166 < @div class="option">< input type="radio" name="q2" value="B"> B) Mars />
167 < @div class="option">< input type="radio" name="q2" value="C"> C) Jupiter />
168 < @div class="option">< input type="radio" name="q2" value="D"> D) Venus />
169 < /div>
170 < /div>
171 < @div class="submit-btn" id="submit">Submit />
172 < @div id="result">< /div>
173 < /div>
174 < script>
175 < let timeleft = 60;
176 < const timerElement = document.getElementById("timer");
177 < const submitBtn = document.getElementById("submit");
178 < const resultElement = document.getElementById("result");
179 < let timerInterval;
180
181 < /script>
182

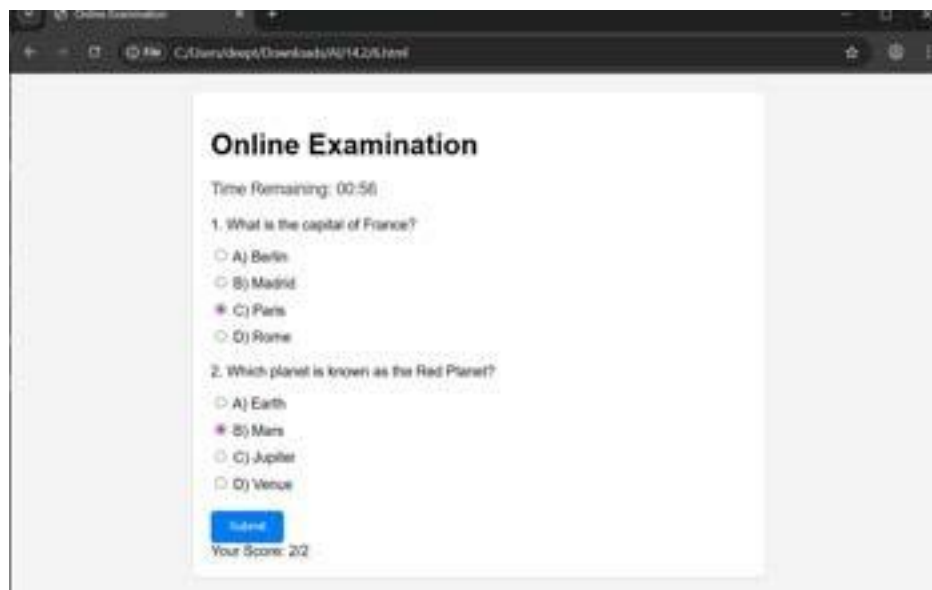
```

```

183 < @head> @head> @head> @style ? % container
184 < @html lang="en">
185 < @body>
186
187 < script>
188 < function startTimer() {
189 < timerInterval = setInterval(() => {
190 < submitBtn.disabled = true;
191 < }, 1000);
192 < }
193
194 < function submitExam() {
195 < submitBtn.disabled = true;
196 < const q1Answer = document.querySelector("input[name='q1']:checked").value;
197 < const q2Answer = document.querySelector("input[name='q2']:checked").value;
198 < let score = 0;
199
200 < if (q1Answer === 'C') score++;
201 < if (q2Answer === 'B') score++;
202 < resultElement.textContent = `Your Score: ${score}/2`;
203 < }
204
205 < submitBtn.addEventListener("click", () => {
206 < clearInterval(timerInterval);
207 < submitExam();
208 < });
209
210 < startTimer();
211 < /script>
212 < /body>
213 < /html>

```

Output :



The screenshot shows a web browser window with the address bar displaying "C:/Users/deep/Downloads/W142/A.html". The page content is titled "Online Examination" and includes a timer showing "Time Remaining: 00:56". There are two questions:

1. What is the capital of France?
☐ A) Berlin
☐ B) Madrid
☒ C) Paris
☐ D) Rome

2. Which planet is known as the Red Planet?
☐ A) Earth
☒ B) Mars
☐ C) Jupiter
☐ D) Venus

Below the questions is a blue "Submit" button and a line indicating "Your Score: 2/2".

Explanation :

- Timer controls exam duration.
- JavaScript manages exam state.
- Prevents multiple submissions.